

MultiMarginLoss#

<https://pytorch.org/docs/stable/generated/torch.nn.MultiMarginLoss.html>

Description:

Creates a criterion that optimizes a multi-class classification hinge loss (margin-based loss) between input `xxx` (a 2D mini-batch Tensor) and output `yyy` (which is a 1D tensor of target class indices, $0 \leq y \leq x.size(1)-1$ $\leq y \leq \text{x.size}(1)-1$): For each mini-batch sample, the loss in terms of the 1D input `xxx` and scalar output `yyy` is: where $i \in \{0, \dots, x.size(0)-1\}$ $i \in \left\{0, \dots, \text{x.size}(0) - 1\right\}$ and $i \neq y_i$ $i \neq y_i = y$. Optionally, you can give non-equal weighting on the classes by passing a 1D weight tensor into the constructor. The loss function then becomes:

Function Signature

```
class torch.nn.MultiMarginLoss(p=1, margin=1.0, weight=None,
size_average=None, reduce=None, reduction='mean')[source]#
```

Parameters

Shape:

```
forward(input, target)[source]#
```

Return type

Returns:

Tensor

Code Examples

```
>>> loss = nn.MultiMarginLoss()
>>> x = torch.tensor([[0.1, 0.2, 0.4, 0.8]])
>>> y = torch.tensor([3])
>>> # 0.25 * ((1-(0.8-0.1)) + (1-(0.8-0.2)) + (1-(0.8-0.4)))
>>> loss(x, y)
tensor(0.32...)
```

Detailed Documentation

Documentation

Creates a criterion that optimizes a multi-class classification hinge loss (margin-based loss) between input `xxx` (a 2D mini-batch Tensor) and output `yyy` (which is a 1D tensor of target class indices, $0 \leq y \leq x.size(1)-1$ $\leq y \leq \text{x.size}(1)-1$):

For each mini-batch sample, the loss in terms of the 1D input `xxx` and scalar output `yyy` is:

where $i \in \{0, \dots, x.size(0)-1\}$ $i \in \left\{0, \dots, \text{x.size}(0) - 1\right\}$ and $i \neq y_i \neq y_i = y$.

Optionally, you can give non-equal weighting on the classes by passing a 1D weight tensor into the constructor.

The loss function then becomes:

`p` (int, optional) – Has a default value of 111. 111 and 222 are the only supported values.

`margin` (float, optional) – Has a default value of 111.

`weight` (Tensor, optional) – a manual rescaling weight given to each class. If given, it has to be a Tensor of size C. Otherwise, it is treated as if having all ones.

`size_average` (bool, optional) – Deprecated (see reduction). By default, the losses are averaged over each loss element in the batch. Note that for some losses, there are multiple elements per sample. If the field `size_average` is set to False, the losses are instead summed for each minibatch. Ignored when `reduce` is False. Default: True

`reduce` (bool, optional) – Deprecated (see reduction). By default, the losses are averaged or summed over observations for each minibatch depending on `size_average`.

When `reduce` is `False`, returns a loss per batch element instead and ignores `size_average`. Default: `True`

`reduction` (str, optional) – Specifies the reduction to apply to the output: 'none' | 'mean' | 'sum'. 'none': no reduction will be applied, 'mean': the sum of the output will be divided by the number of elements in the output, 'sum': the output will be summed. Note: `size_average` and `reduce` are in the process of being deprecated, and in the meantime, specifying either of those two args will override `reduction`. Default: 'mean'

Input: $(N,C)(N, C)(N,C)$ or $(C)(C)(C)$, where NNN is the batch size and CCC is the number of classes.

Target: $(N)(N)(N)$ or $()()()$, where each value is $0 \leq \text{targets}[i] \leq C-1$.

Output: scalar. If reduction is 'none', then same shape as the target.

Runs the forward pass.